

ラボ 4.3: PowerShell を使用したベースライン設定

1. はじめに

1.1. 必要なリソース

- Windows 10 仮想マシン
- このラボは、Windows 10を実行している場合はホストマシンでも実行できます。

1.2. まとめ

このラボでは、PowerShellを使用してシステムの基本的なセキュリティベースラインを作成する方法を学びます。既にこの作業の経験がある場合は、PowerShellで収集する方法がわからないWindowsの設定について考えて、調べてみることをお勧めします

1.3. 目標

- PowerShellの基本を学ぶ
 - Get-Helpコマンドレット
 - Select-Stringによる出力のフィルタリング
- 基本的なベースラインを収集する方法を学ぶ
 - インストールされているすべてのサービスを一覧表示する
 - すべてのネットワーク接続を一覧表示する
 - すべてのローカルユーザーを一覧表示する
 - インストールされているすべてのソフトウェアを一覧表示する

2. PowerShellの基礎

多くの場合、PowerShellはcmd.exeと同じように使用できます。出力は通常cmd.exeとは少し異なりますが、ほとんどのcmd.exeコマンドは使用できます

まずは PowerShell、Start Menu

2.1. cmd.exe の機能

PowerShell で以下の cmd.exe コマンドを試してください。

```
PS C:\Windows\student> cd \Users\Public
```

Public ディレクトリをユーザーのホームディレクトリに変更します。

✓ 成功

新しいフォルダが表示され、プロンプトの表示が変わります。

```
PS C:\Users\Public>
```

```
PS C:\Users\Public> echo "My text file" > ICS410.txt
```

ICS410.txt というテキスト ファイルにテキストをエコーします。

✓ 成功

ニュースがないのは良いニュースです... 出力は表示されないはずです。

```
PS C:\Users\Public> type ICS410.txt
```

ICS410.txt の内容を画面に入力します。

✓ 成功

```
My text file
```

```
PS C:\Users\Public> copy ICS410.txt ICS410-2.txt; dir
```

ICS410.txt を同様の名前を持つ 2 番目のファイルにコピーし、ディレクトリ内のファイルを一覧表示します。

✓ 成功

次のようなものになるはずです:

Directory: C:\Users\Public

Mode	LastWriteTime	Length	Name
d-r--	8/24/2023 4:33 PM		Documents
d-r--	7/17/2023 4:27 PM		Downloads
d-r--	7/17/2023 4:27 PM		Music
d-r--	7/17/2023 4:27 PM		Pictures
d-r--	7/17/2023 4:27 PM		Videos
-a----	8/24/2023 6:39 PM	30	ICS410-2.txt
-a----	8/24/2023 6:39 PM	30	ICS410.txt

注意:日付とファイル名は上記の例と若干異なる場合があります。

```
PS C:\Users\Public> del ICS410-2.txt; dir
```

新しいファイルのコピーを削除し、ディレクトリ内のファイルを一覧表示します。

✓ 成功

Directory: C:\Users\Public

Mode	LastWriteTime	Length	Name
d-r--	8/24/2023 4:33 PM		Documents
d-r--	7/17/2023 4:27 PM		Downloads
d-r--	7/17/2023 4:27 PM		Music
d-r--	7/17/2023 4:27 PM		Pictures
d-r--	7/17/2023 4:27 PM		Videos
-a----	8/24/2023 6:39 PM	30	ICS410.txt

注意:日付とファイル名は上記の例と若干異なる場合があります。

2.2. Linux ライクな機能

LinuxユーザーにとってPowerShellで最初に魅力的なのは、基本的なbashコマンドのエイリアスです

PowerShellで次の疑似Linuxコマンドを試してみましょう

```
PSC:\Users\Public> cd ~
```

現在のユーザー（生徒）のホームディレクトリ/フォルダに変更します

Linuxと同様に、`~` ホームフォルダには特別な意味があります。 `\Users\student`

✓ 成功

コマンドプロンプトが変更されます。

```
PS C:\Users\student>
```

```
PSC:\Users\student> ls > file1.txt
```

ディレクトリ内のファイルを一覧表示し、`file1.txt` として保存します。

✓ 成功

出力は期待されません

```
PSC:\Users\student> ls > file2.txt
```

ディレクトリ内のファイルを再度一覧表示し、`file2.txt` として保存します。

✓ 成功

出力は期待されません


```
PSC:\Users\student> cat file1.txt, file2.txt
```

Get-Content 両方のファイルの内容を連結して画面に表示します。ここではカンマが必要ですが、実際の Linux bash ターミナルではカンマは不要であることに注意してください。これは、cat コマンドが、カンマで区切られたファイル名を必要とする PowerShell コマンドのエイリアスにすぎないためです

✓ 成功

Directory: C:\Users\student

Mode	LastWriteTime	Length	Name
d-----	4/28/2023 2:02 PM		.idlerc
d-----	6/3/2023 11:33 PM		.vscode
d-----	6/4/2023 12:01 AM		.zenmap
d-r---	7/18/2023 8:34 PM		3D Objects
d-r---	7/18/2023 8:34 PM		Contacts
d-r---	8/13/2023 1:16 PM		Desktop
d-r---	7/18/2023 8:34 PM		Documents
d-r---	7/18/2023 8:34 PM		Downloads
d-r---	7/18/2023 8:34 PM		Favorites
d-r---	7/18/2023 8:34 PM		Links
d-r---	7/18/2023 8:34 PM		Music
d-r---	7/18/2023 8:34 PM		Pictures
d-r---	7/18/2023 8:34 PM		Saved Games
d-r---	7/18/2023 8:34 PM		Searches
d-r---	7/18/2023 8:34 PM		Videos
-a----	8/24/2023 6:54 PM	0	file1.txt

Directory: C:\Users\student

Mode	LastWriteTime	Length	Name
d-----	4/28/2023 2:02 PM		.idlerc
d-----	6/3/2023 11:33 PM		.vscode
d-----	6/4/2023 12:01 AM		.zenmap
d-r---	7/18/2023 8:34 PM		3D Objects
d-r---	7/18/2023 8:34 PM		Contacts
d-r---	8/13/2023 1:16 PM		Desktop
d-r---	7/18/2023 8:34 PM		Documents
d-r---	7/18/2023 8:34 PM		Downloads
d-r---	7/18/2023 8:34 PM		Favorites
d-r---	7/18/2023 8:34 PM		Links
d-r---	7/18/2023 8:34 PM		Music
d-r---	7/18/2023 8:34 PM		Pictures
d-r---	7/18/2023 8:34 PM		Saved Games
d-r---	7/18/2023 8:34 PM		Searches
d-r---	7/18/2023 8:34 PM		Videos
-a----	8/24/2023 6:54 PM	5416	file1.txt
-a----	8/24/2023 6:59 PM	0	file2.txt

注意:日付とファイル名は上記の例と若干異なる場合があります。

```
PSC:\Users\student> man diff
```

diffコマンドのヘルプページを表示します。これにより、2つのファイルの違いを比較できます。Linuxのdiffコマンドに慣れている方は、オプション（そして出力）がかなり異なることに気付くでしょう。

✓ 成功

```
NAME
    Compare-Object

SYNTAX
    Compare-Object [-ReferenceObject] <psobject[]> [-DifferenceObject] <psobject[]> [-SyncWindow <int>]
    [-Property
    <Object[]>] [-ExcludeDifferent] [-IncludeEqual] [-PassThru] [-Culture <string>] [-CaseSensitive]
    [<CommonParameters>]

ALIASES
    compare
    diff

REMARKS
    Get-Help cannot find the Help files for this cmdlet on this computer. It is displaying only partial
    help.

    -- To download and install Help files for the module that includes this cmdlet, use Update-Help.
    -- To view the Help topic for this cmdlet online, type: "Get-Help Compare-Object -Online" or
    go to https://go.microsoft.com/fwlink/?LinkID=113286.
```

```
PSC:\Users\student> diff (gc file1.txt) (gc file2.txt)
```

作成した2つのファイルの違いを表示します。

✓ 成功

```
InputObject
-----
-a----      8/24/2023    6:54 PM          5416 file1.txt
-a----      8/24/2023    6:59 PM              0 file2.txt
-a----      8/24/2023    6:54 PM              0 file1.txt
```

注意:日付とファイル名は上記の例と若干異なる場合があります。

```
PSC:\Users\student> ps
```

実行中のプロセスを表示します。

✓ 成功

Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
-----	-----	-----	-----	-----	--	--	-----
... <output removed> ...							
283	17	4888	12544		10416	0	svchost
3069	0	200	24		4	0	System
312	41	8744	14576	0.44	5644	1	taskhostw
106	6	1212	5368		4160	0	Windows.WARP.JITService
156	11	1304	956		588	0	wininit
271	12	2700	2376		628	1	winlogon
521	36	15036	364	0.25	11136	1	WinStore.App

```
PSC:\Users\student> clear
```

ターミナル ウィンドウの内容をクリアします。

Linuxユーザーにとって、これは素晴らしいスタートです。残念ながら、エイリアスはそれ以上の機能は備えておらず、コマンドもLinuxで慣れ親しんだような便利なオプションをすべてサポートしているわけではありません。とはいえ、少なくとも、誤って入力してしまう可能性のある一般的なコマンドのほとんどに対して、意味のある応答が得られるのは確かです。

2.3. PowerShell ヘルプ

Get-Help コマンドレットを使用すると、利用可能なコマンドレットを検索し、それらのコマンドレットのヘルプページを表示できます。Get-Help を単独で入力すると、コマンドの使い方が表示されます

```
PSC:\Users\student> Get-Help
```

✓ 成功

TOPIC

Windows PowerShell Help System

SHORT DESCRIPTION

Displays help about Windows PowerShell cmdlets and concepts.

... 簡潔にするために出力は削除されました ...

EXAMPLES:

```
Save-Help           : Download help files from the Internet and saves
                      them on a file share.
Update-Help         : Downloads and installs help files from the
                      Internet or a file share.
Get-Help Get-Process : Displays help about the Get-Process cmdlet.
Get-Help Get-Process -Online
                      : Opens online help for the Get-Process cmdlet.
Help Get-Process    : Displays help about Get-Process one page at a time.
Get-Process -?      : Displays help about the Get-Process cmdlet.
Get-Help About_Modules : Displays help about Windows PowerShell modules.
Get-Help remoting    : Searches the help topics for the word "remoting."
```

SEE ALSO:

```
about_Updatable_Help
Get-Help
Save-Help
Update-Help
```

ワイルドカードを使うと、ヘルプページがあるコマンドレットを確認できます。ワイルドカードは、アスタリスク文字を単独で使用することも、文字列と組み合わせて部分一致を検索することもできます。

```
PSC:\Users\student> Get-Help *
```

利用可能な PowerShell コマンドレットの非常に長いリストを表示します。

`more` 以下に示すようにパイプすると、**スペースバー**を使用して一度に 1 ページずつスクロールでき、**q** 一番下までスクロールしたくない場合は終了の文字を使用できます。

```
PSC:\Users\student> Get-Help * | more
```

同じ長いリストですが、一度に 1 ページずつ表示します。

Get-Help には、代わりに使用できるエイリアスが2つあります。上記のコマンドは、省略形の **help** コマンド または **man** (Linux/Unix版で**manual**から取得) を使用して実行できます。どちらのエイリアスにも が含まれています **more**。これらのエイリアスを使用すると、ヘルプシステムをより簡単に使用できます。

Get-Help コマンドを使用して部分一致を検索してみましょう。

```
PSC:\Users\student> help Set*
```

これにより、すべてのコマンドレットが表示されます **Set-**。

```
PSC:\Users\student> help Get-Net*
```

で始まるすべてのコマンドレットを表示します。 **Get-Net** そのほとんどはネットワークを扱います。

PowerShellの使い方を学ぶ上で、helpコマンドはあなたの心の支えになります。ぜひ活用してください！

2.4. 出力のフィルタリング

PowerShellには、他のPowerShellコマンドレットの出力をフィルタリングするために使用できるコマンドがいくつかあります **Select-**。使用するコマンドは、フィルタリングする出力によって異なります

Selectで始まるコマンドレットはごくわずかです。1つ使ってみましょう。

```
PSC:\Users\student> help Select*
```

利用可能なさまざまな選択コマンドを表示します

✓ 成功

Name	Category	Module	Synopsis
----	-----	-----	-----
select	Alias		Select-Object
Select-Object	Cmdlet	Microsoft.PowerShell.U...	...
Select-String	Cmdlet	Microsoft.PowerShell.U...	...
Select-Xml	Cmdlet	Microsoft.PowerShell.U...	...

Linuxに詳しい方なら、Select-StringコマンドレットはLinuxのgrepコマンドと同様に動作します。ASCIIまたはUTF-8テキストファイルの内容のような単純なテキストストリームで動作します

```
PSC:\Users\student> help Select-String
```

Select-String のヘルプページを表示します。

✓ 成功

NAME

Select-String

SYNTAX

```
Select-String [-Pattern] <string[]> [-Path] <string[]> [-SimpleMatch] [-CaseSensitive] [-Quiet] [-List] [-Include <string[]>] [-Exclude <string[]>] [-NotMatch] [-AllMatches] [-Encoding {unicode | utf7 | utf8 | utf32 | ascii | bigendianunicode | default | oem}] [-Context <int[]>] [<CommonParameters>]
```

```
Select-String [-Pattern] <string[]> -InputObject <psobject> [-SimpleMatch] [-CaseSensitive] [-Quiet] [-List] [-Include <string[]>] [-Exclude <string[]>] [-NotMatch] [-AllMatches] [-Encoding {unicode | utf7 | utf8 | utf32 | ascii | bigendianunicode | default | oem}] [-Context <int[]>] [<CommonParameters>]
```

```
Select-String [-Pattern] <string[]> -LiteralPath <string[]> [-SimpleMatch] [-CaseSensitive] [-Quiet] [-List] [-Include <string[]>] [-Exclude <string[]>] [-NotMatch] [-AllMatches] [-Encoding {unicode | utf7 | utf8 | utf32 | ascii | bigendianunicode | default | oem}] [-Context <int[]>] [<CommonParameters>]
```

ALIASES

sls

コマンドレットを使ってテキストファイルをフィルタリング（Linux用語ではgrep）してみましょう。また、上記にリストされているエイリアス `Select-String` も使用します `Select-String sls`

```
PSC:\Users\student> type file1.txt | sls Favorite
```

の内容を印刷し `file1.txt`、単語が含まれる行のみを表示します `Favorite`。

💡 ヒント

`type`、`cat` は `gc` すべて `Get-Content` コマンドレットのエイリアスです。 `type` は、これを行うためのDOSの従来のコマンドです

✓ 成功

d-r---

7/18/2023

8:34 PM

Favorites

```
PSC:\Users\student> gc file1.txt | sls ^d
```

ファイルリストからディレクトリ（フォルダ）のみを出力します。これらは文字で始まるディレクトリです `d`。この `^` 文字はキャレット記号と呼ばれ、各行の先頭を表します。を含めない場合は、行のどこかに `^` その文字が含まれるすべての行が出力されます。 `d`

🔥 ヒント

`gc` および `Get-Content` に `sls` は短いエイリアスを使用していることに注意してください `Select-String`。

✓ 成功

d-r---	7/18/2023	8:34 PM	3D Objects
d-r---	7/18/2023	8:34 PM	Contacts
d-r---	8/13/2023	1:16 PM	Desktop
d-r---	7/18/2023	8:34 PM	Documents
d-r---	7/18/2023	8:34 PM	Downloads
d-r---	7/18/2023	8:34 PM	Favorites
d-r---	7/18/2023	8:34 PM	Links
d-r---	7/18/2023	8:34 PM	Music
d-r---	7/18/2023	8:34 PM	Pictures
d-r---	7/18/2023	8:34 PM	Saved Games
d-r---	7/18/2023	8:34 PM	Searches
d-r---	7/18/2023	8:34 PM	Videos

注意:日付とファイル名は上記の例と若干異なる場合があります。

次に、コマンドとそのエイリアス以外のもので試してみましょう `Get-Content`。

```
PSC:\Users\student> dir | sls Favorite
```

✓ 成功

Favorites			
file1.txt:17:d-r---	7/18/2023	8:34 PM	Favorites
file2.txt:17:d-r---	7/18/2023	8:34 PM	Favorites

注意:日付とファイル名は上記の例と若干異なる場合があります。

`dir` コマンドを使ってパイプでに渡すと `sls`、一見するとうまくいったように見えますが、よく見ると3つの結果が返されています。コマンド `dir` 自体には、文字列を含む行が1行しか表示されません `Favorites`。

少し違う例を試してみましょう。 `^d` 先ほど使用した文字列を使って、トップレベルのディレクトリ/フォルダをすべて検索してみましょう。


```
PSC:\Users\student> dir | sls ^d
```

✓ 成功

```
Desktop
Documents
Downloads
file1.txt:8:d----- 4/28/2023 2:02 PM .idlerc
file1.txt:9:d----- 6/3/2023 11:33 PM .vscode
file1.txt:10:d----- 6/4/2023 12:01 AM .zenmap
file1.txt:11:d-r--- 7/18/2023 8:34 PM 3D Objects
file1.txt:12:d-r--- 7/18/2023 8:34 PM Contacts
file1.txt:14:d-r--- 8/13/2023 1:16 PM Desktop
file1.txt:15:d-r--- 7/18/2023 8:34 PM Documents
file1.txt:16:d-r--- 7/18/2023 8:34 PM Downloads
file1.txt:17:d-r--- 7/18/2023 8:34 PM Favorites
file1.txt:19:d-r--- 7/18/2023 8:34 PM Links
file1.txt:20:d-r--- 7/18/2023 8:34 PM Music
file1.txt:22:d-r--- 7/18/2023 8:34 PM Pictures
file1.txt:23:d-r--- 7/18/2023 8:34 PM Saved Games
file1.txt:24:d-r--- 7/18/2023 8:34 PM Searches
file1.txt:25:d-r--- 7/18/2023 8:34 PM Videos
file2.txt:8:d----- 4/28/2023 2:02 PM .idlerc
file2.txt:9:d----- 6/3/2023 11:33 PM .vscode
file2.txt:10:d----- 6/4/2023 12:01 AM .zenmap
file2.txt:11:d-r--- 7/18/2023 8:34 PM 3D Objects
file2.txt:12:d-r--- 7/18/2023 8:34 PM Contacts
file2.txt:14:d-r--- 8/13/2023 1:16 PM Desktop
file2.txt:15:d-r--- 7/18/2023 8:34 PM Documents
file2.txt:16:d-r--- 7/18/2023 8:34 PM Downloads
file2.txt:17:d-r--- 7/18/2023 8:34 PM Favorites
file2.txt:19:d-r--- 7/18/2023 8:34 PM Links
file2.txt:20:d-r--- 7/18/2023 8:34 PM Music
file2.txt:22:d-r--- 7/18/2023 8:34 PM Pictures
file2.txt:23:d-r--- 7/18/2023 8:34 PM Saved Games
file2.txt:24:d-r--- 7/18/2023 8:34 PM Searches
file2.txt:25:d-r--- 7/18/2023 8:34 PM Videos
```

注意:日付とファイル名は上記の例と若干異なる場合があります。

よく見ると、`type` との出力は にパイプしない限りほぼ同じであるにもかかわらず、コマンド `type` を `dir` 使用した場合の出力とは異なることに気付くでしょう `Select-String`。これは、一部のコマンドレットはすべてをテキストオブジェクトとして出力するのに対し、他のコマンドレットはプログラムオブジェクトとして出力するためです。コマンド `sls` はファイル名だけでなく、このディレクトリにある2つのテキストファイル (`file1.txt`と`file2.txt`) の内容も解析しています。Linux/Unixコマンドプロンプトに慣れている方にとって、これは大きな考え方の転換となるでしょう。BashやZshのようなシェルではなく、PythonやRubyのようなプログラミング言語のようにアプローチする必要があります。

3. 共通ベースライン

PowerShell を少し使用したので、Windows 10 VM で共通ベースラインを収集して、通常の状態を確認してみましょう。ベースラインを長期間収集することで、マシン上で何が変更されたかを判断するのに役立ち、セキュリティ侵害の強力な指標となる可能性があります

3.1. サービスの実行

このマシンで実行されているすべてのサービスを一覧表示するコマンドレットを検索してください。Get-Service というオプションがすぐに見つかるはずです。そのエイリアスは gsv です。

```
PSC:\Users\student> help *service*
```

✓ 成功

Name	Category	Module	Synopsis
----	-----	-----	-----
Get-Service	Cmdlet	Microsoft.PowerShell.M...	...
New-Service	Cmdlet	Microsoft.PowerShell.M...	...
New-WebServiceProxy	Cmdlet	Microsoft.PowerShell.M...	...
Restart-Service	Cmdlet	Microsoft.PowerShell.M...	...
Resume-Service	Cmdlet	Microsoft.PowerShell.M...	...
Set-Service	Cmdlet	Microsoft.PowerShell.M...	...
Start-Service	Cmdlet	Microsoft.PowerShell.M...	...
Stop-Service	Cmdlet	Microsoft.PowerShell.M...	...
Suspend-Service	Cmdlet	Microsoft.PowerShell.M...	...
Set-NetFirewallServiceFilter	Function	NetSecurity	...
Get-NetFirewallServiceFilter	Function	NetSecurity	...

コマンド `Get-Service` レットは良い出発点になりそうです。ヘルプページを見てみましょう

```
PSC:\Users\student> help Get-Service
```

✓ 成功

NAME

Get-Service

SYNTAX

```
Get-Service [[-Name] <string[]>] [-ComputerName <string[]>] [-DependentServices] [-RequiredServices] [-Include <string[]>] [-Exclude <string[]>] [<CommonParameters>]
```

```
Get-Service -DisplayName <string[]> [-ComputerName <string[]>] [-DependentServices] [-RequiredServices] [-Include <string[]>] [-Exclude <string[]>] [<CommonParameters>]
```

```
Get-Service [-ComputerName <string[]>] [-DependentServices] [-RequiredServices] [-Include <string[]>] [-Exclude <string[]>] [-InputObject <ServiceController[]>] [<CommonParameters>]
```

ALIASES

gsv

Get-Service またはそのエイリアスを単独で実行してみてください。実行中と停止中の両方のサービスがすべて表示されることに注目してください。

```
PSC:\Users\student> gsv
```

✓ 成功

Status	Name	DisplayName
-----	----	-----
Stopped	AarSvc_415b1	Agent Activation Runtime_415b1
Stopped	AJRouter	AllJoyn Router Service
Stopped	ALG	Application Layer Gateway Service
Stopped	AppIDSvc	Application Identity
Stopped	Appinfo	Application Information
Stopped	AppMgmt	Application Management
Stopped	AppReadiness	App Readiness
Stopped	AppVClient	Microsoft App-V Client
Stopped	AppXSvc	AppX Deployment Service (AppXSVC)
Stopped	AssignedAccessM...	AssignedAccessManager Service
Running	AudioEndpointBu...	Windows Audio Endpoint Builder
Running	Audiosrv	Windows Audio
Stopped	autotimesvc	Cellular Time
Stopped	AxInstSV	ActiveX Installer (AxInstSV)
Stopped	BcastDVRUserSer...	GameDVR and Broadcast User Service_...
Stopped	BDESVC	BitLocker Drive Encryption Service
Running	BFE	Base Filtering Engine

...出力は続きます...

sls を使って実行中のサービスだけを表示してみましょう。すぐに何も返されないことに気づくでしょう。まさに私たちが求めている結果ではありません。

```
PSC:\Users\student> gsv | sls Running
```

✓ 成功

出力がありません...

Select-String アプローチは、`Get-Service` オブジェクトを出力するため、コマンドレットでは失敗します。

それでは、出力をもう少し詳しく見てみましょう `Get-Service`。ヘルプページに `-Property` 有効な構文が記載されていないことに注目してください。つまり、この出力をフィルタリングする簡単な方法がないということです。つまり、`Where-Object` 出力フィールド（オブジェクトプロパティと呼ばれます）に基づいてフィルタリングするには、コマンドレットを使用する必要があります。プロパティのリストを取得するには `Get-Service`、まずコマンドレットを使用する必要があります `Get-Member`。ヘルプページを見てみましょう。

```
PSC:\Users\student> help Get-Member
```

✓ 成功

NAME

Get-Member

SYNTAX

```
Get-Member [[-Name] <string[]>] [-InputObject <psobject>] [-MemberType {AliasProperty | CodeProperty  
| Property | NoteProperty | ScriptProperty | Properties | PropertySet |  
Method | CodeMethod | ScriptMethod | Methods | ParameterizedProperty | MemberSet | Event | Dynamic |  
All}] [-View {Extended | Adapted | Base | All}] [-Static] [-Force]  
[<CommonParameters>]
```

ALIASES

gm

`Get-Service` したがって、の出力をパイプすると、出力される `Get-Member` サービス オブジェクトの詳細のリストを取得できます `Get-Service`。

PSC:\Users\student> gsv | gm

✓ 成功

TypeName: System.ServiceProcess.ServiceController

Name	MemberType	Definition
-----	-----	-----
Name	AliasProperty	Name = ServiceName
RequiredServices	AliasProperty	RequiredServices = ServicesDependedOn
Disposed	Event	System.EventHandler Disposed(System.Object, System.EventArgs)
Close	Method	void Close()
Continue	Method	void Continue()
CreateObjRef	Method	System.Runtime.Remoting.ObjRef CreateObjRef(type requestedType)
Dispose	Method	void Dispose(), void IDisposable.Dispose()
Equals	Method	bool Equals(System.Object obj)
ExecuteCommand	Method	void ExecuteCommand(int command)
GetHashCode	Method	int GetHashCode()
GetLifetimeService	Method	System.Object GetLifetimeService()
GetType	Method	type GetType()
InitializeLifetimeService	Method	System.Object InitializeLifetimeService()
Pause	Method	void Pause()
Refresh	Method	void Refresh()
Start	Method	void Start(), void Start(string[] args)
Stop	Method	void Stop()
WaitForStatus	Method	void WaitForStatus(System.ServiceProcess.ServiceControllerStatus desiredStatus), void WaitForStatus(System.ServiceProcess.ServiceControl...
CanPauseAndContinue	Property	bool CanPauseAndContinue {get;}
CanShutdown	Property	bool CanShutdown {get;}
CanStop	Property	bool CanStop {get;}
Container	Property	System.ComponentModel.IContainer Container {get;}
DependentServices	Property	System.ServiceProcess.ServiceController[] DependentServices {get;}
DisplayName	Property	string DisplayName {get;set;}
MachineName	Property	string MachineName {get;set;}
ServiceHandle	Property	System.Runtime.InteropServices.SafeHandle ServiceHandle {get;}
ServiceName	Property	string ServiceName {get;set;}
ServicesDependedOn	Property	System.ServiceProcess.ServiceController[] ServicesDependedOn {get;}
ServiceType	Property	System.ServiceProcess.ServiceType ServiceType {get;}
Site	Property	System.ComponentModel.ISite Site {get;set;}
StartType	Property	System.ServiceProcess.ServiceStartMode StartType {get;}
Status	Property	System.ServiceProcess.ServiceControllerStatus Status {get;}
ToString	ScriptMethod	System.Object ToString();

の出力を にパイプして、 `Get-Service` で利用可能なすべてのおよびを `Get-Member` 一覧表示します。 の出力に、サービスが実行中かどうかを示すという列があったことを思い出してください。上記の出力から、は有効なプロパティであることがわかりま

す。つまり、でこれを使用することで、実行中のすべてのプロセスをフィルタリングできるはずです。 `Get-Service.Methods()` `Get-Service.Properties` `Get-Service` `Get-Service Status` `Status` `Where-Object`

では、の使い方を考えてみましょう `Where-Object`。

```
PSC:\Users\student> help Where-Object
```


✓ 成功

NAME

`Where-Object`

SYNTAX

```
Where-Object [-Property] <string> [[-Value] <Object>] [-InputObject <psobject>] [-EQ]
[<CommonParameters>]

Where-Object [-FilterScript] <scriptblock> [-InputObject <psobject>] [<CommonParameters>]

Where-Object [-Property] <string> [[-Value] <Object>] -Match [-InputObject <psobject>]
[<CommonParameters>]

Where-Object [-Property] <string> [[-Value] <Object>] -CEQ [-InputObject <psobject>]
[<CommonParameters>]

Where-Object [-Property] <string> [[-Value] <Object>] -NE [-InputObject <psobject>]
[<CommonParameters>]

Where-Object [-Property] <string> [[-Value] <Object>] -CNE [-InputObject <psobject>]
[<CommonParameters>]

Where-Object [-Property] <string> [[-Value] <Object>] -GT [-InputObject <psobject>]
[<CommonParameters>]

Where-Object [-Property] <string> [[-Value] <Object>] -CGT [-InputObject <psobject>]
[<CommonParameters>]

Where-Object [-Property] <string> [[-Value] <Object>] -LT [-InputObject <psobject>]
[<CommonParameters>]

Where-Object [-Property] <string> [[-Value] <Object>] -CLT [-InputObject <psobject>]
[<CommonParameters>]

Where-Object [-Property] <string> [[-Value] <Object>] -GE [-InputObject <psobject>]
[<CommonParameters>]

Where-Object [-Property] <string> [[-Value] <Object>] -CGE [-InputObject <psobject>]
[<CommonParameters>]

Where-Object [-Property] <string> [[-Value] <Object>] -LE [-InputObject <psobject>]
[<CommonParameters>]

Where-Object [-Property] <string> [[-Value] <Object>] -CLE [-InputObject <psobject>]
[<CommonParameters>]

Where-Object [-Property] <string> [[-Value] <Object>] -Like [-InputObject <psobject>]
[<CommonParameters>]

Where-Object [-Property] <string> [[-Value] <Object>] -CLike [-InputObject <psobject>]
[<CommonParameters>]
```

```
Where-Object [-Property] <string> [[-Value] <Object>] -NotLike [-InputObject <psobject>]
[<CommonParameters>]

Where-Object [-Property] <string> [[-Value] <Object>] -CNotLike [-InputObject <psobject>]
[<CommonParameters>]

Where-Object [-Property] <string> [[-Value] <Object>] -CMatch [-InputObject <psobject>]
[<CommonParameters>]

Where-Object [-Property] <string> [[-Value] <Object>] -NotMatch [-InputObject <psobject>]
[<CommonParameters>]

Where-Object [-Property] <string> [[-Value] <Object>] -CNotMatch [-InputObject <psobject>]
[<CommonParameters>]

Where-Object [-Property] <string> [[-Value] <Object>] -Contains [-InputObject <psobject>]
[<CommonParameters>]

Where-Object [-Property] <string> [[-Value] <Object>] -CContains [-InputObject <psobject>]
[<CommonParameters>]

Where-Object [-Property] <string> [[-Value] <Object>] -NotContains [-InputObject <psobject>]
[<CommonParameters>]

Where-Object [-Property] <string> [[-Value] <Object>] -CNotContains [-InputObject <psobject>]
[<CommonParameters>]

Where-Object [-Property] <string> [[-Value] <Object>] -In [-InputObject <psobject>]
[<CommonParameters>]

Where-Object [-Property] <string> [[-Value] <Object>] -CIn [-InputObject <psobject>]
[<CommonParameters>]

Where-Object [-Property] <string> [[-Value] <Object>] -NotIn [-InputObject <psobject>]
[<CommonParameters>]

Where-Object [-Property] <string> [[-Value] <Object>] -CNotIn [-InputObject <psobject>]
[<CommonParameters>]

Where-Object [-Property] <string> [[-Value] <Object>] -Is [-InputObject <psobject>]
[<CommonParameters>]

Where-Object [-Property] <string> [[-Value] <Object>] -IsNot [-InputObject <psobject>]
[<CommonParameters>]

ALIASES
where
?
```

まず、エイリアスがであることに注目してください `where`。 `Where-Object` 構文としては、使用できる比較演算子が多数ありますが、最初のものが `-eq` 機能するはずです。

ここで、 の出力をパイプして `Get-Service`、フィルターに使用する `Where-Object` ものを指定します。 `Get-Service.Property`

```
PSC:\Users\student> gsv | where Status -eq Running
```

✓ 成功

Status	Name	DisplayName
-----	----	-----
Running	AudioEndpointBu...	Windows Audio Endpoint Builder
Running	Audiosrv	Windows Audio
Running	BFE	Base Filtering Engine
Running	BITS	Background Intelligent Transfer Ser...
Running	BrokerInfrastru...	Background Tasks Infrastructure Ser...
Running	BthAvctpSvc	AVCTP service

...出力は続きます...

構文が正しければ、実行中のサービスの一覧が表示されます。これを修正して、停止中のサービスのみを表示してみましょう。次に、サービス名を検索して試してみてください。

3.2. ネットワーク接続

ネットワーク接続の一覧表示は簡単です。 `Get-NetTCPConnection` コマンドレットを使用すると、すべてのネットワーク接続を一覧表示できます。まずはヘルプページを見てみましょう

```
PSC:\Users\student> help Get-NetTCPConnection
```

✓ 成功

NAME

Get-NetTCPConnection

SYNTAX

```
Get-NetTCPConnection [[-LocalAddress] <string[]>] [[-LocalPort] <uint16[]>] [-RemoteAddress  
<string[]>] [-RemotePort <uint16[]>] [-State {Closed | Listen | SynSent |  
SynReceived | Established | FinWait1 | FinWait2 | CloseWait | Closing | LastAck | TimeWait |  
DeleteTCB | Bound}] [-AppliedSetting {Internet | Datacenter | Compat |  
DatacenterCustom | InternetCustom}] [-OwningProcess <uint32[]>] [-CreationTime <datetime[]>] [-  
OffloadState {InHost | Offloading | Offloaded | Uploading}] [-CimSession  
<CimSession[]>] [-ThrottleLimit <int>] [-AsJob] [<CommonParameters>]
```

ALIASES

None

単独で実行してみて、何が得られるかを確認してみましょう。

```
PSC:\Users\student> Get-NetTCPConnection
```

✓ 成功



LocalAddress	LocalPort	RemoteAddress	RemotePort	State
AppliedSetting	OwningProcess			
-----	-----	-----	-----	-----
::	7680	::	0	Listen
2708				
::	1541	::	0	Listen
704				
::	1540	::	0	Listen
2664				
::	1539	::	0	Listen
1240				
::	1538	::	0	Listen
1252				
::	1537	::	0	Listen
588				
::	1536	::	0	Listen
712				
::	445	::	0	Listen
4				
::	135	::	0	Listen
980				
0.0.0.0	1553	0.0.0.0	0	Bound
8080				
0.0.0.0	1551	0.0.0.0	0	Bound
8080				
0.0.0.0	1549	0.0.0.0	0	Bound
8080				
0.0.0.0	1547	0.0.0.0	0	Bound
8080				
127.0.0.1	15292	0.0.0.0	0	Listen
9820				
0.0.0.0	5040	0.0.0.0	0	Listen
5976				
127.0.0.1	1553	127.0.0.1	1552	Established
Internet	8080			
127.0.0.1	1552	127.0.0.1	1553	Established
Internet	8080			
127.0.0.1	1551	127.0.0.1	1550	Established
Internet	8080			
127.0.0.1	1550	127.0.0.1	1551	Established
Internet	8080			
127.0.0.1	1549	127.0.0.1	1548	Established
Internet	8080			
127.0.0.1	1548	127.0.0.1	1549	Established
Internet	8080			
127.0.0.1	1547	127.0.0.1	1546	Established
Internet	8080			
127.0.0.1	1546	127.0.0.1	1547	Established
Internet	8080			
0.0.0.0	1541	0.0.0.0	0	Listen
704				

0.0.0.0	1540	0.0.0.0	0	Listen
2664				
0.0.0.0	1539	0.0.0.0	0	Listen
1240				
0.0.0.0	1538	0.0.0.0	0	Listen
1252				
0.0.0.0	1537	0.0.0.0	0	Listen
588				
0.0.0.0	1536	0.0.0.0	0	Listen
712				
169.254.148.237	139	0.0.0.0	0	Listen
4				
0.0.0.0	135	0.0.0.0	0	Listen
980				

ヘルプページを見ると、`Get-NetTCPConnection` いくつかのオプションがサポートされていることがわかります。いくつか試してみしましょう。

PSC:\Users\student> Get-NetTCPConnection -State Listen

✓ 成功

LocalAddress AppliedSetting ----- -----	LocalPort	RemoteAddress -----	RemotePort -----	State
:: 2708	7680	::	0	Listen
:: 704	1541	::	0	Listen
:: 2664	1540	::	0	Listen
:: 1240	1539	::	0	Listen
:: 1252	1538	::	0	Listen
:: 588	1537	::	0	Listen
:: 712	1536	::	0	Listen
:: 4	445	::	0	Listen
:: 980	135	::	0	Listen
127.0.0.1 9820	15292	0.0.0.0	0	Listen
0.0.0.0 5976	5040	0.0.0.0	0	Listen
0.0.0.0 704	1541	0.0.0.0	0	Listen
0.0.0.0 2664	1540	0.0.0.0	0	Listen
0.0.0.0 1240	1539	0.0.0.0	0	Listen
0.0.0.0 1252	1538	0.0.0.0	0	Listen
0.0.0.0 588	1537	0.0.0.0	0	Listen
0.0.0.0 712	1536	0.0.0.0	0	Listen
169.254.148.237 4	139	0.0.0.0	0	Listen
0.0.0.0 980	135	0.0.0.0	0	Listen


```
PSC:\Users\student> Get-NetTCPConnection -LocalPort 135
```

✓ 成功

LocalAddress AppliedSetting OwningProcess ----- -----	LocalPort	RemoteAddress -----	RemotePort	State
:: 980	135	::	0	Listen
0.0.0.0 980	135	0.0.0.0	0	Listen

3.3. ローカルユーザー

場合によっては、PowerShellオブジェクトをそのまま使用して必要なデータを取得するのが非常に難しいことがあります。マシン上で利用可能なユーザーのリストはその一例です。PowerShellでこれを行う方法を知っていれば `wmi.exe`、コマンドレットを使用してPowerShellでそのデータを活用できます `Get-WmiObject`。ヘルプページをご覧ください

```
PSC:\Users\student> help Get-WmiObject
```

✓ 成功

NAME

Get-WmiObject

SYNTAX

```
Get-WmiObject [-Class] <string> [[-Property] <string[]>] [-Filter <string>] [-Amended] [-DirectRead]
[-AsJob] [-Impersonation {Default | Anonymous | Identify | Impersonate |
Delegate}] [-Authentication {Default | None | Connect | Call | Packet | PacketIntegrity |
PacketPrivacy | Unchanged}] [-Locale <string>] [-EnableAllPrivileges] [-Authority
<string>] [-Credential <pscredential>] [-ThrottleLimit <int>] [-ComputerName <string[]>] [-Namespace
<string>] [<CommonParameters>]
```

```
Get-WmiObject [[-Class] <string>] [-Recurse] [-Amended] [-List] [-AsJob] [-Impersonation {Default |
Anonymous | Identify | Impersonate | Delegate}] [-Authentication {Default
| None | Connect | Call | Packet | PacketIntegrity | PacketPrivacy | Unchanged}] [-Locale <string>]
[-EnableAllPrivileges] [-Authority <string>] [-Credential <pscredential>]
[-ThrottleLimit <int>] [-ComputerName <string[]>] [-Namespace <string>] [<CommonParameters>]
```

```
Get-WmiObject -Query <string> [-Amended] [-DirectRead] [-AsJob] [-Impersonation {Default | Anonymous
| Identify | Impersonate | Delegate}] [-Authentication {Default | None |
Connect | Call | Packet | PacketIntegrity | PacketPrivacy | Unchanged}] [-Locale <string>] [-
EnableAllPrivileges] [-Authority <string>] [-Credential <pscredential>]
[-ThrottleLimit <int>] [-ComputerName <string[]>] [-Namespace <string>] [<CommonParameters>]
```

```
Get-WmiObject [-Amended] [-AsJob] [-Impersonation {Default | Anonymous | Identify | Impersonate |
Delegate}] [-Authentication {Default | None | Connect | Call | Packet |
PacketIntegrity | PacketPrivacy | Unchanged}] [-Locale <string>] [-EnableAllPrivileges] [-Authority
<string>] [-Credential <pscredential>] [-ThrottleLimit <int>]
[-ComputerName <string[]>] [-Namespace <string>] [<CommonParameters>]
```

```
Get-WmiObject [-Amended] [-AsJob] [-Impersonation {Default | Anonymous | Identify | Impersonate |
Delegate}] [-Authentication {Default | None | Connect | Call | Packet |
PacketIntegrity | PacketPrivacy | Unchanged}] [-Locale <string>] [-EnableAllPrivileges] [-Authority
<string>] [-Credential <pscredential>] [-ThrottleLimit <int>]
[-ComputerName <string[]>] [-Namespace <string>] [<CommonParameters>]
```

ALIASES

gwm

オプションに気づきましたか `-List` ? 試してみてください。ただし、more にパイプしない場合は、少し時間がかかります。more にパイプする場合は、`SPACEBAR` 一度に 1 ページずつスクロールするには `q` more を終了するには `q` と入力してください。

```
PSC:\Users\student> gwm -List | more
```

✓ 成功

Name	Methods	Properties
CIM_Indication	{}	{CorrelatedIndications, IndicationFilterName, IndicationIdentifier, IndicationTime...}
CIM_ClassIndication	{}	{ClassDefinition, CorrelatedIndications, IndicationFilterName, IndicationIdentifier...}
CIM_ClassDeletion	{}	{ClassDefinition, CorrelatedIndications, IndicationFilterName, IndicationIdentifier...}
CIM_ClassCreation	{}	{ClassDefinition, CorrelatedIndications, IndicationFilterName, IndicationIdentifier...}
CIM_ClassModification	{}	{ClassDefinition, CorrelatedIndications, IndicationFilterName, IndicationIdentifier...}
CIM_InstIndication	{}	{CorrelatedIndications, IndicationFilterName, IndicationIdentifier, IndicationTime...}
CIM_InstCreation	{}	{CorrelatedIndications, IndicationFilterName, IndicationIdentifier, IndicationTime...}
CIM_InstModification	{}	{CorrelatedIndications, IndicationFilterName, IndicationIdentifier, IndicationTime...}
CIM_InstDeletion	{}	{CorrelatedIndications, IndicationFilterName, IndicationIdentifier, IndicationTime...}

...出力は続きます...

-Listは優れたオプションですが、出力が長すぎて読めません。ユーザーに関連する項目で出力を `Select-String` フィルタリングするには、`gwm -List` を使用しましょう

```
PSC:\Users\student> gwmi -List | sls User
```

✓ 成功

```
\\SANS-C3B81VIT3L\ROOT\cimv2:CIM_UserDevice
\\SANS-C3B81VIT3L\ROOT\cimv2:Win32_UserAccount
\\SANS-C3B81VIT3L\ROOT\cimv2:Win32_UserProfile
\\SANS-C3B81VIT3L\ROOT\cimv2:__NTLMUser9X
\\SANS-C3B81VIT3L\ROOT\cimv2:Win32_OfflineFilesUserConfiguration
\\SANS-C3B81VIT3L\ROOT\cimv2:Win32_UserDesktop
\\SANS-C3B81VIT3L\ROOT\cimv2:Win32_RoamingProfileUserConfiguration
\\SANS-C3B81VIT3L\ROOT\cimv2:Win32_VolumeUserQuota
\\SANS-C3B81VIT3L\ROOT\cimv2:Win32_RoamingUserHealthConfiguration
\\SANS-C3B81VIT3L\ROOT\cimv2:Win32_UserStateConfigurationControls
\\SANS-C3B81VIT3L\ROOT\cimv2:Win32_SystemUsers
\\SANS-C3B81VIT3L\ROOT\cimv2:Win32_UserInDomain
\\SANS-C3B81VIT3L\ROOT\cimv2:Win32_GroupUser
\\SANS-C3B81VIT3L\ROOT\cimv2:Win32_FolderRedirectionUserConfiguration
\\SANS-C3B81VIT3L\ROOT\cimv2:Win32_LoggedOnUser
\\SANS-C3B81VIT3L\ROOT\cimv2:Win32_NTLogEventUser
\\SANS-C3B81VIT3L\root\cimv2:Win32_PerfFormattedData_LSM_UserInputDelayperProcess
\\SANS-C3B81VIT3L\root\cimv2:Win32_PerfRawData_LSM_UserInputDelayperProcess
\\SANS-C3B81VIT3L\root\cimv2:Win32_PerfFormattedData_LSM_UserInputDelayperSession
\\SANS-C3B81VIT3L\root\cimv2:Win32_PerfRawData_LSM_UserInputDelayperSession
```

幸いなことに、Select-String コマンドレットはここでうまく機能しました。いくつかのオプションがありますが、すべて試してみると、`Win32_UserAccount` 利用可能なユーザーの短いリストを表示するにはこれが最適であることがわかります。ヘルプページで学んだことを活かして、オブジェクト `-class` だけを表示するオプションを使ってみましょう `Win32_UserAccount`。

```
PSC:\Users\student> gwmim -Class Win32_UserAccount
```

✓ 成功

```
AccountType : 512
Caption      : SANS-C3B81VIT3L\Administrator
Domain       : SANS-C3B81VIT3L
SID          : S-1-5-21-641609421-2599998773-1805479120-500
FullName     :
Name         : Administrator

AccountType : 512
Caption      : SANS-C3B81VIT3L\DefaultAccount
Domain       : SANS-C3B81VIT3L
SID          : S-1-5-21-641609421-2599998773-1805479120-503
FullName     :
Name         : DefaultAccount

AccountType : 512
Caption      : SANS-C3B81VIT3L\defaultuser0
Domain       : SANS-C3B81VIT3L
SID          : S-1-5-21-641609421-2599998773-1805479120-1000
FullName     :
Name         : defaultuser0

AccountType : 512
Caption      : SANS-C3B81VIT3L\Guest
Domain       : SANS-C3B81VIT3L
SID          : S-1-5-21-641609421-2599998773-1805479120-501
FullName     :
Name         : Guest

AccountType : 512
Caption      : SANS-C3B81VIT3L\student
Domain       : SANS-C3B81VIT3L
SID          : S-1-5-21-641609421-2599998773-1805479120-1001
FullName     :
Name         : student

AccountType : 512
Caption      : SANS-C3B81VIT3L\WDAGUtilityAccount
Domain       : SANS-C3B81VIT3L
SID          : S-1-5-21-641609421-2599998773-1805479120-504
FullName     :
Name         : WDAGUtilityAccount
```

3.4. インストールされたソフトウェア

それでは、インストールされたソフトウェアで動作するWMIクラスを探してみましょう。クラス名に「ソフトウェア」が含まれるクラスを検索することもできますが、Microsoftはインストールされたソフトウェアに対して「製品」という用語を使用することを推奨しています

```
PSC:\Users\student> gwmiclass -List | sls Product
```

✓ 成功

```
\\SANS-C3B81VIT3L\R00T\cimv2:CIM_Product  
\\SANS-C3B81VIT3L\R00T\cimv2:Win32_ComputerSystemProduct  
\\SANS-C3B81VIT3L\R00T\cimv2:Win32_Product  
\\SANS-C3B81VIT3L\R00T\cimv2:Win32_ProductResource  
\\SANS-C3B81VIT3L\R00T\cimv2:CIM_ProductSoftwareFeatures  
\\SANS-C3B81VIT3L\R00T\cimv2:Win32_ProductSoftwareFeatures  
\\SANS-C3B81VIT3L\R00T\cimv2:CIM_ProductFRU  
\\SANS-C3B81VIT3L\R00T\cimv2:CIM_FRUIncludesProduct  
\\SANS-C3B81VIT3L\R00T\cimv2:CIM_ProductPhysicalElements  
\\SANS-C3B81VIT3L\R00T\cimv2:CIM_ProductSupport  
\\SANS-C3B81VIT3L\R00T\cimv2:SoftwareLicensingProduct  
\\SANS-C3B81VIT3L\R00T\cimv2:Win32_ProductCheck  
\\SANS-C3B81VIT3L\R00T\cimv2:CIM_ProductParentChild  
\\SANS-C3B81VIT3L\R00T\cimv2:CIM_ProductProductDependency  
\\SANS-C3B81VIT3L\R00T\cimv2:CIM_CompatibleProduct
```

Win32_Productクラスは簡単に見つけられるはずです。これは問題なく動作します。ただし、Cim_Productクラスも存在することに注意してください。試してみると、リストに64ビット専用アプリケーションがいくつか追加されるかもしれません。

```
PSC:\Users\student> gwmf -Class Win32_Product
```

✓ 成功

```
IdentifyingNumber : VMware-56 4d b5 b9 16 3b 61 7f-87 da 6c f7 c6 9e 80 a8
Name               : VMware Virtual Platform
Vendor             : VMware, Inc.
Version            : None
Caption            : Computer System Product
```

```
IdentifyingNumber : {A0258B41-0D21-496B-A342-B8BCCB8F2B8D}
Name               : Python 3.8.3 Core Interpreter (64-bit)
Vendor             : Python Software Foundation
Version            : 3.8.3150.0
Caption            : Python 3.8.3 Core Interpreter (64-bit)
```

```
IdentifyingNumber : {698BFA23-9AF5-43B1-A08E-293477F8FD9B}
Name               : Python 3.8.3 pip Bootstrap (64-bit)
Vendor             : Python Software Foundation
Version            : 3.8.3150.0
Caption            : Python 3.8.3 pip Bootstrap (64-bit)
```

```
IdentifyingNumber : {13E05234-E037-4C96-BF0C-585FD0A8E2B0}
Name               : Python 3.8.3 Add to Path (64-bit)
Vendor             : Python Software Foundation
Version            : 3.8.3150.0
Caption            : Python 3.8.3 Add to Path (64-bit)
```

...出力は続きます...

```
PSC:\Users\student> gwmi -Class Cim_Product
```

✓ 成功

```
IdentifyingNumber : {A0258B41-0D21-496B-A342-B8BCCB8F2B8D}  
Name               : Python 3.8.3 Core Interpreter (64-bit)  
Vendor            : Python Software Foundation  
Version           : 3.8.3150.0  
Caption           : Python 3.8.3 Core Interpreter (64-bit)
```

```
IdentifyingNumber : {698BFA23-9AF5-43B1-A08E-293477F8FD9B}  
Name               : Python 3.8.3 pip Bootstrap (64-bit)  
Vendor            : Python Software Foundation  
Version           : 3.8.3150.0  
Caption           : Python 3.8.3 pip Bootstrap (64-bit)
```

```
IdentifyingNumber : {13E05234-E037-4C96-BF0C-585FD0A8E2B0}  
Name               : Python 3.8.3 Add to Path (64-bit)  
Vendor            : Python Software Foundation  
Version           : 3.8.3150.0  
Caption           : Python 3.8.3 Add to Path (64-bit)
```

...出力は続きます...

4. ボーナスチャレンジ

他の参加者のラボの完了を待っている場合、またはこのラボで紹介された概念をさらに深く探求したい場合は、以下のアイデアを試してみてください。これらのチャレンジにはチュートリアルは用意されていませんが、行き詰まった場合は、コース作成者のジャスティンに取り組みの成果を示せば、喜んでヒントを教えてください。justin [@controlthings.io](mailto:justin@controlthings.io)までメールでお問い合わせください

ベースラインの構築と検証に役立つ一般的なEDRツールをダウンロードしてインストールしてみてください。

- [ヴェロキラプトル](#)
- [カンザス](#)